# LLM大模型部署实战指南：Ollama简化流程，OpenLLM灵活部署，LocalAI本地优化，Dify赋能应用开发

Original 汀丶人工智能 汀丶人工智能 2024年08月14日 19:31

## LLM大模型部署实战指南：Ollama简化流程，OpenLLM灵活部署，LocalAI本地优化，Dify赋能应用开发

## 1. Ollama 部署的本地模型(🔺)

Ollama 是一个开源框架，专为在本地机器上便捷部署和运行大型语言模型（LLM）而设计。，这是 Ollama 的官网地址：https://ollama.com/




以下是其主要特点和功能概述：

1. 简化部署：Ollama 目标在于简化在 Docker 容器中部署大型语言模型的过程，使得非专业用户也能方便地管理和运行这些复杂的模型。
2. 轻量级与可扩展：作为轻量级框架，Ollama 保持了较小的资源占用，同时具备良好的可扩展性，允许用户根据需要调整配置以适应不同规模的项目和硬件条件。

3 API支持：提供了一个简洁的 API，使得开发者能够轻松创建、运行和管理大型语言模型实例，降低了与模型交互的技术门槛。

4 预构建模型库：包含一系列预先训练好的大型语言模型，用户可以直接选用这些模型应用于自己的应用程序，无需从头训练或自行寻找模型源




1.1 一键安装

curl: (77) error setting certificate verify locations:CAfile: /data/usr/local/anaconda/ssl/cacert.pemCApath: none

报错原因：cacert.pem 的寻址路径 CAfile 不对，也就是在该路径下找不到文件。

解决方法：

1 找到你的 cacert.pem 文件所在位置 /path/to/cacert.pem。如果你没有该证书，可以先在 https://curl.se/ca/cacert.pem 下载，保存在某个目录中。

2 设置环境变量

```
export CURL_CA_BUNDLE=/path/to/cacert.pem
#将"/path/to/cacert.pem"替换为你的证书文件的实际路径。
export CURL_CA_BUNDLE=/www/anaconda3/anaconda3/ssl/cacert.pem
```

执行下载

```
curl -fsSL https://ollama.com/install.sh | sh
```

### 1.2 手动安装

ollama中文网：https://ollama.fan/getting-started/linux/

1 下载 ollama 二进制文件：Ollama 以自包含的二进制文件形式分发。将其下载到您的 PATH 中的目录：

```
sudo curl -L https://ollama.com/download/ollama-linux-amd64 -o /usr/bin/ollam

sudo chmod +x /usr/bin/ollama
```

1 将 Ollama 添加为启动服务（推荐）：为 Ollama 创建一个用户：

```
sudo useradd -r -s /bin/false -m -d /usr/share/ollama ollama
```

3.在 /etc/systemd/system/ollama.service 中创建一个服务文件：

```
#vim ollama.service

[Unit]

Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/bin/ollama serve
User=ollama
Group=ollama
Restart=always
RestartSec=3

[Install]
WantedBy=default.target
```

1 然后启动服务：

```
sudo systemctl daemon-reload
sudo systemctl enable ollama
```

1 启动 Ollama¶
使用 systemd 启动 Ollama：

```
sudo systemctl start ollama
```

1 更新，查看日志

```
#再次运行
sudo curl -L https://ollama.com/download/ollama-linux-amd64 -o /usr/bin/ollam
sudo chmod +x /usr/bin/ollama

#要查看作为启动服务运行的 Ollama 的日志，请运行：
journalctl -u ollama
```

1 步骤7：关闭 Ollama 服务

```
#关闭ollama服务
service ollama stop
```

## 1.3 Linux内网离线安装Ollama

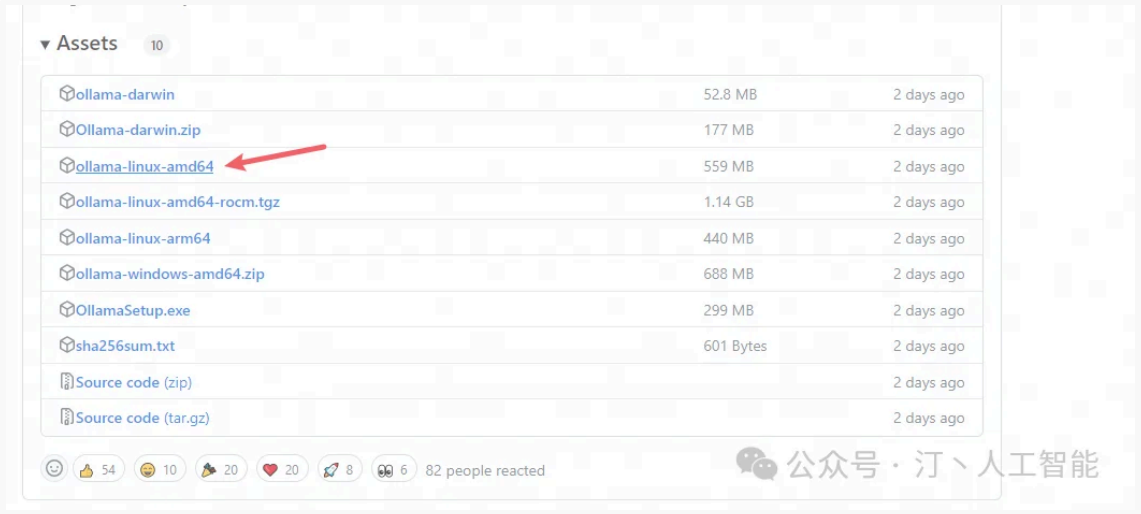
1 查看服务器CPU的型号

```
##查看Linux系统CPU型号命令，我的服务器cpu型号是x86_64
lscpu
```

1 步骤2：根据CPU型号下载Ollama安装包，并保存到目录

下载地址：https://github.com/ollama/ollama/releases/

```
#x86_64 CPU选择下载ollama-linux-amd64
#aarch64|arm64 CPU选择下载ollama-linux-arm64
```




```
#有网机器下载过来也一样
wget https://ollama.com/download/ollama-linux-amd64
```

下载到离线服务器上:/usr/bin/ollama  ollama就是你下载的ollama-linux-amd64 改名了(mv)，其他步骤一致

### 1.4 修改存储路径

Ollama模型默认存储在：

macOS: ~/.ollama/models
Linux: /usr/share/ollama/.ollama/models
Windows: C:\Users<username>.ollama\models

如果 Ollama 作为 systemd 服务运行，则应使用以下命令设置环境变量systemctl：

1

通过调用 来编辑 systemd 服务systemctl edit ollama.service。这将打开一个编辑器。

2

Environment对于每个环境变量，在部分下添加一行[Service]：

直接在“/etc/systemd/system/ollama.service”增了2行:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0:7861"
Environment="OLLAMA_MODELS=/www/algorithm/LLM_model/models"
```

1

保存并退出。

2

重新加载systemd并重新启动 Ollama：

```
systemctl daemon-reload
systemctl restart ollama
```

参考链接：https://github.com/ollama/ollama/blob/main/docs/faq.md

1 使用 systemd 启动 Ollama：

```
sudo systemctl start ollama
```

1 终止

终止(ollama加载的大模型将会停止占用显存，此时ollama属于失联状态，部署和运行操作失效，会报错：

Error: could not connect to ollama app, is it running?需要启动后，才可以进行部署和运行操作

```
systemctl stop ollama.service
```

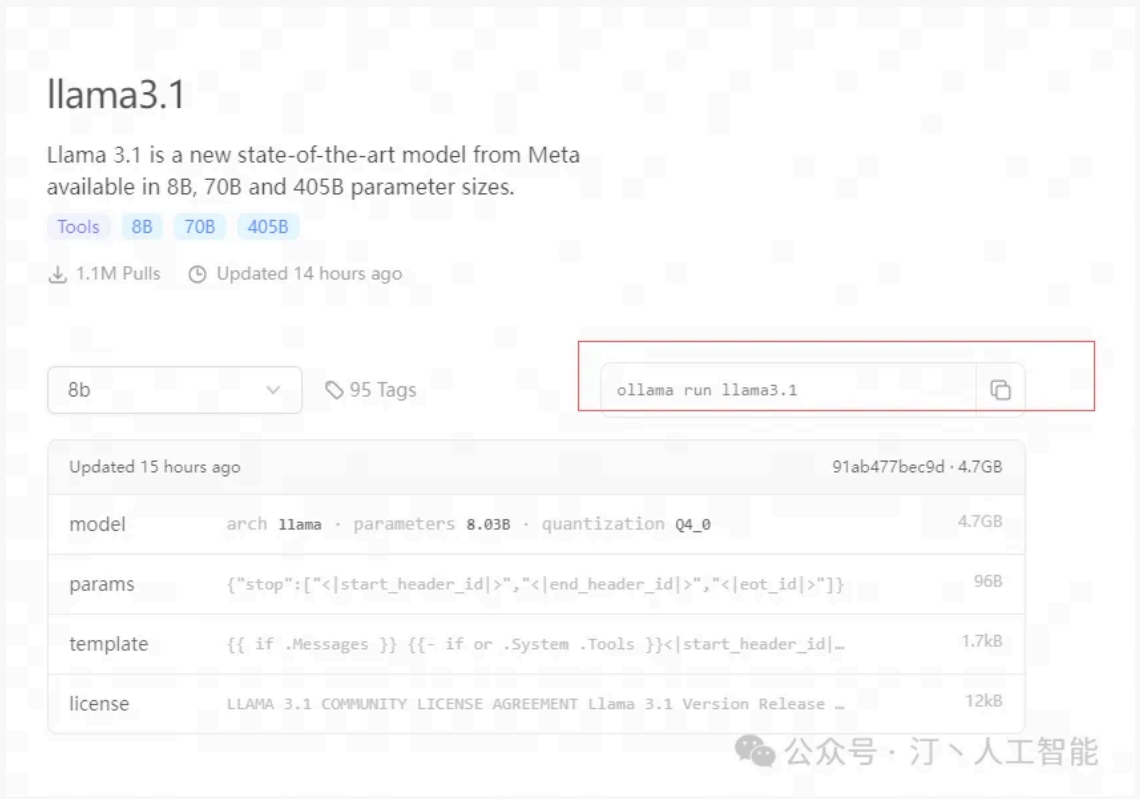
终止后启动（启动后，可以接着使用ollama 部署和运行大模型）

```
systemctl start ollama.service
```

## 1.5 启动LLM

下载模型

```
ollama pull llama3.1
ollama pull qwen2
```




运行大模型

```
ollama run llama3.1
ollama run qwen2
```




查看是否识别到大模型: `ollama list` ，如果成功, 则会看到大模型

```
ollama list
NAME            ID              SIZE    MODIFIED
qwen2:latest    e0d4e1163c58    4.4 GB  3 hours ago
```

使用该 `ollama ps` 命令查看当前已加载到内存中的模型。

```
NAME            ID              SIZE    PROCESSOR       UNTIL
qwen2:latest    e0d4e1163c58    5.7 GB  100% GPU        3 minutes from now
```

nvidia-smi查看

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.86.10              Driver Version: 535.86.10    CUDA Version
|-----------------------------------------+----------------------+-----------
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile U
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util
|                                         |                      |
|=========================================+======================+===========
|   0  Tesla V100-SXM2-32GB           On  | 00000000:00:08.0 Off |
| N/A   35C    P0              56W / 300W |   5404MiB / 32768MiB |      0%
|                                         |                      |
+-----------------------------------------+----------------------+-----------

+-----------------------------------------------------------------------------+
| Processes:
|  GPU   GI   CI        PID   Type   Process name
|        ID   ID
|=============================================================================
|    0   N/A  N/A   3062036      C   ...unners/cuda_v11/ollama_llama_server
+-----------------------------------------------------------------------------+
```

启动后，我们可验证是否可用：

```
curl http://10.80.2.195:7861/api/chat -d '{
  "model": "llama3.1",
  "messages": [
    { "role": "user", "content": "why is the sky blue?" }
  ]
}'
```

## 1.6 更多其他配置

**Ollama 可以设置的环境变量**：

`OLLAMA_HOST`：这个变量定义了Ollama监听的网络接口。通过设置OLLAMA_HOST=0.0.0.0，我们可以让Ollama监听所有可用的网络接口，从而允许外部网络访问。

OLLAMA_MODELS：这个变量指定了模型镜像的存储路径。通过设置OLLAMA_MODELS=F:\OllamaCache，我们可以将模型镜像存储在E盘，避免C盘空间不足的问题。

OLLAMA_KEEP_ALIVE：这个变量控制模型在内存中的存活时间。设置OLLAMA_KEEP_ALIVE=24h可以让模型在内存中保持24小时，提高访问速度。

OLLAMA_PORT：这个变量允许我们更改Ollama的默认端口。例如，设置OLLAMA_PORT=8080可以将服务端口从默认的11434更改为8080。

OLLAMA_NUM_PARALLEL：这个变量决定了Ollama可以同时处理的用户请求数量。设置OLLAMA_NUM_PARALLEL=4可以让Ollama同时处理两个并发请求。

OLLAMA_MAX_LOADED_MODELS：这个变量限制了Ollama可以同时加载的模型数量。设置OLLAMA_MAX_LOADED_MODELS=4可以确保系统资源得到合理分配。

Environment=“OLLAMA_PORT=9380”没有用

这样指定：`Environment="OLLAMA_HOST=0.0.0.0:7861"`

指定 GPU
本地有多张 GPU，如何用指定的 GPU 来运行 Ollama？在Linux上创建如下配置文件，并配置环境变量 CUDA_VISIBLE_DEVICES 来指定运行 Ollama 的 GPU，再重启 Ollama 服务即可【测试序号从0还是1开始，应是从0开始】。

```
vim /etc/systemd/system/ollama.service
[Service]
Environment="CUDA_VISIBLE_DEVICES=0,1"
```

### 1.7 Ollama常见命令

1 重启 ollama

```
systemctl daemon-reload
systemctl restart ollama
```

1 重启 ollama 服务

ubuntu/debian

```
sudo apt update
sudo apt install lsof
stop ollama
lsof -i :11434
kill <PID>
ollama serve
```

Ubuntu

```
sudo apt update
sudo apt install lsof
stop ollama
lsof -i :11434
kill <PID>
ollama serve
```

1 确认服务端口状态：

```
netstat -tulpn | grep 11434
```

1 配置服务

为使外网环境能够访问到服务，需要对 HOST 进行配置。

打开配置文件：

```
vim /etc/systemd/system/ollama.service
```

根据情况修改变量 Environment：

服务器环境下：

```
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

虚拟机环境下：

```
Environment="OLLAMA_HOST=服务器内网IP地址:11434"
```

### 1.8 卸载Ollama

如果决定不再使用Ollama，可以通过以下步骤将其完全从系统中移除：

（1）停止并禁用服务：

```
sudo systemctl stop ollama
sudo systemctl disable ollama
```

（2）删除服务文件和Ollama二进制文件：

```
sudo rm /etc/systemd/system/ollama.service
sudo rm $(which ollama)
```

（3）清理Ollama用户和组：

```
sudo rm -r /usr/share/ollama
sudo userdel ollama
sudo groupdel ollama
```

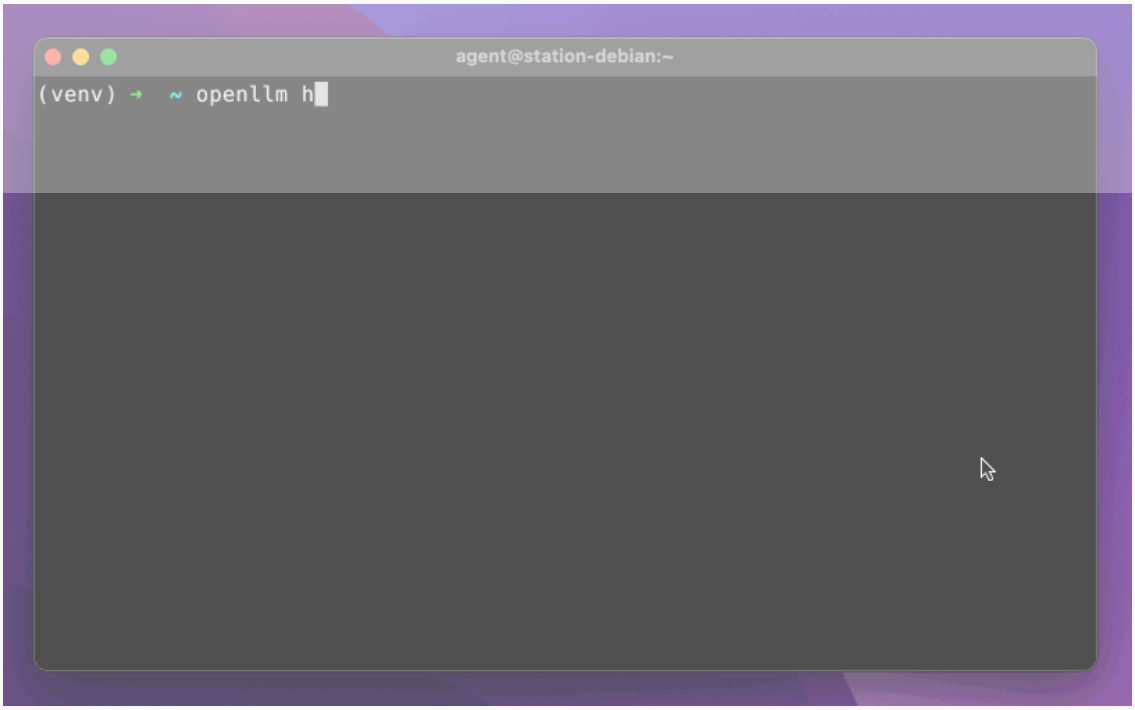
通过以上步骤，不仅能够在Linux平台上成功安装和配置Ollama，还能够灵活地进行更新和卸载。

广告


大模型应用开发极简入门：基于GPT-4和ChatGPT LangChain LLM应用开发蛇尾书（图灵出品）
京东 京东配送
¥29.9
购买

## 2.OpenLLM 部署

OpenLLM 于 2023 年 6 月开源，是一个用于部署大语言模型的框架。目前，该项目在GitHub 上已经获得了 9.6K星标。其最初的口号是通过一行代码或相对轻松地在不同的大语言模型之间切换，为个人用户提供方便。OpenLLM是一个用于在生产环境中操作大型语言模型（LLM）的开放平台，它可以轻松地微调、服务、部署和监控任何LLM。




安装

```
pip install openllm  # or pip3 install openllm
openllm hello
```

支持模型

- Llama-3.1
- Llama-3
- Phi-3
- Mistral
- Gemma-2
- Qwen-2
- Gemma
- Llama-2
- Mixtral



在 设置 > 模型供应商 > OpenLLM 中填入：

模型名称：

服务器 URL：http://<Machine_IP>:3333 替换成您的机器 IP 地址“保存”后即可在应用中使用该模型。

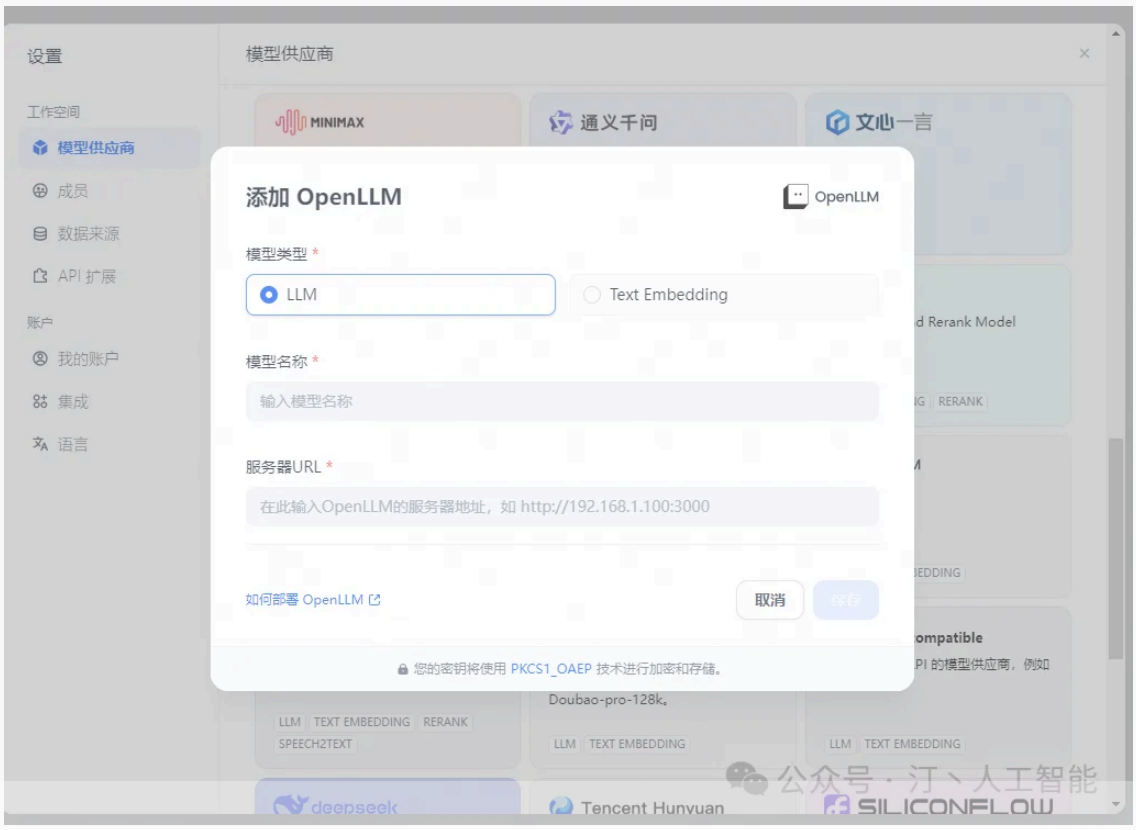
OpenLLM 提供了一个内置的 Python 客户端，允许您与模型进行交互。在不同的终端窗口或 Jupyter notebook 中，创建一个客户端以开始与模型交互：

```
import openllm
client = openllm.client.HTTPClient('http://localhost:3000')
client.query('Explain to me the difference between "further" and "farther"')
```

可以使用 openllm query 命令从终端查询模型：

```
export OPENLLM_ENDPOINT=http://localhost:3000
openllm query 'Explain to me the difference between "further" and "farther"'
```

使用 openllm models 命令查看 OpenLLM 支持的模型及其变体列表。



## 3.LocalAI 部署

LocalAI 是一个本地推理框架，提供了 RESTFul API，与 OpenAI API 规范兼容。它允许你在消费级硬件上本地或者在自有服务器上运行 LLM（和其他模型），支持与 ggml 格式兼容的多种模型家族。不需要 GPU。Dify 支持以本地部署的方式接入 LocalAI 部署的大型语言模型推理和 embedding 能力。

giuhub：https://github.com/mudler/LocalAI/tree/master

官网手册：https://localai.io/docs/getting-started/models/

官方快速部署手册案例：https://localai.io/docs/getting-started/models/

1 首先拉取 LocalAI 代码仓库，并进入指定目录

```
git clone https://github.com/go-skynet/LocalAI
cd LocalAI/examples/langchain-chroma
```

1 下载demo LLM 和 Embedding 模型（仅供参考）

```
wget https://huggingface.co/skeskinen/ggml/resolve/main/all-MiniLM-L6-v2/ggml
wget https://gpt4all.io/models/ggml-gpt4all-j.bin -O models/ggml-gpt4all-j
```

参考文章：告别Hugging Face模型下载难题

1 配置 .env 文件

```
mv .env.example .env
```

NOTE：请确保 .env 中的 THREADS 变量值不超过您本机的 CPU 核心数。

1 启动 LocalAI

```
#start with docker-compose
$docker-compose up -d --build

#tail the logs & wait until the build completes
docker logs -f langchain-chroma-api-1
7:16AM INF Starting LocalAI using 4 threads, with models path: /models
7:16AM INF LocalAI version: v1.24.1 (9cc8d9086580bd2a96f5c96a6b873242879c70bc

 ┌───────────────────────────────────────────────────┐
 │                   Fiber v2.48.0                   │
 │               http://127.0.0.1:8080               │
 │       (bound on host 0.0.0.0 and port 8080)       │
 │                                                   │
 │ Handlers ............. 55  Processes ........... 1 │
```

```
| Prefork ....... Disabled  PID ................ 14 |
```

开放了本机 http://127.0.0.1:8080 作为 LocalAI 请求 API 的端点。

并提供了两个模型，分别为：

LLM 模型：ggml-gpt4all-j

对外访问名称：gpt-3.5-turbo（该名称可自定义，在 models/gpt-3.5-turbo.yaml 中配置。

Embedding 模型：all-MiniLM-L6-v2

对外访问名称：text-embedding-ada-002（该名称可自定义，在 models/embeddings.yaml 中配置。

使用 Dify Docker 部署方式的需要注意网络配置，确保 Dify 容器可以访问到localAI 的端点，Dify 容器内部无法访问到 localhost，需要使用宿主机 IP 地址。

1 LocalAI API 服务部署完毕，在 Dify 中使用接入模型

在 设置 > 模型供应商 > LocalAI 中填入：

模型 1：ggml-gpt4all-j

模型类型：文本生成
模型名称：gpt-3.5-turbo
服务器 URL：http://127.0.0.1:8080
若 Dify 为 docker 部署，请填入 host 域名：http://your-LocalAI-endpoint-domain:8080，可填写局域网 IP 地址，如：http://192.168.1.100:8080

模型 2：all-MiniLM-L6-v2

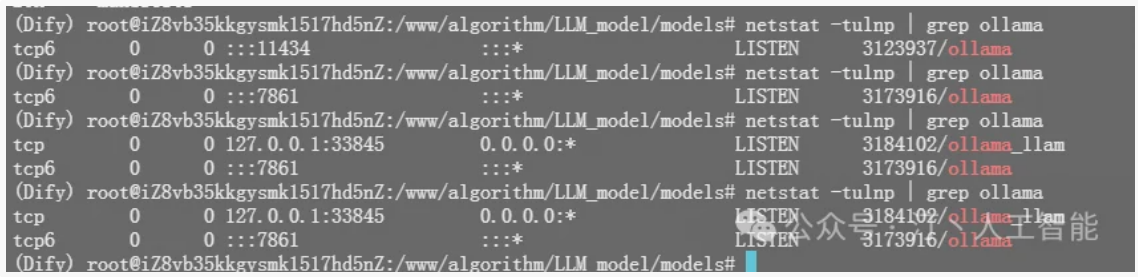
模型类型：Embeddings
模型名称：text-embedding-ada-002
服务器 URL：http://127.0.0.1:8080
若 Dify 为 docker 部署，请填入 host 域名：http://your-LocalAI-endpoint-domain:8080，可填写局域网 IP 地址，如：http://192.168.1.100:8080

如需获取 LocalAI 更多信息，请参考：https://github.com/go-skynet/LocalAI

## 4.配置LLM+Dify(ollama 🔺)

确认服务端口状态：

```
netstat -tulnp | grep ollama
#netstat -tulpn | grep 11434
```




报错：“Error: could not connect to ollama app, is it running?”

参考链接：https://stackoverflow.com/questions/78437376/run-ollama-run-llama3-in-colab-raise-err-error-could-not-connect-to-ollama

/etc/systemd/system/ollama.service文件是：

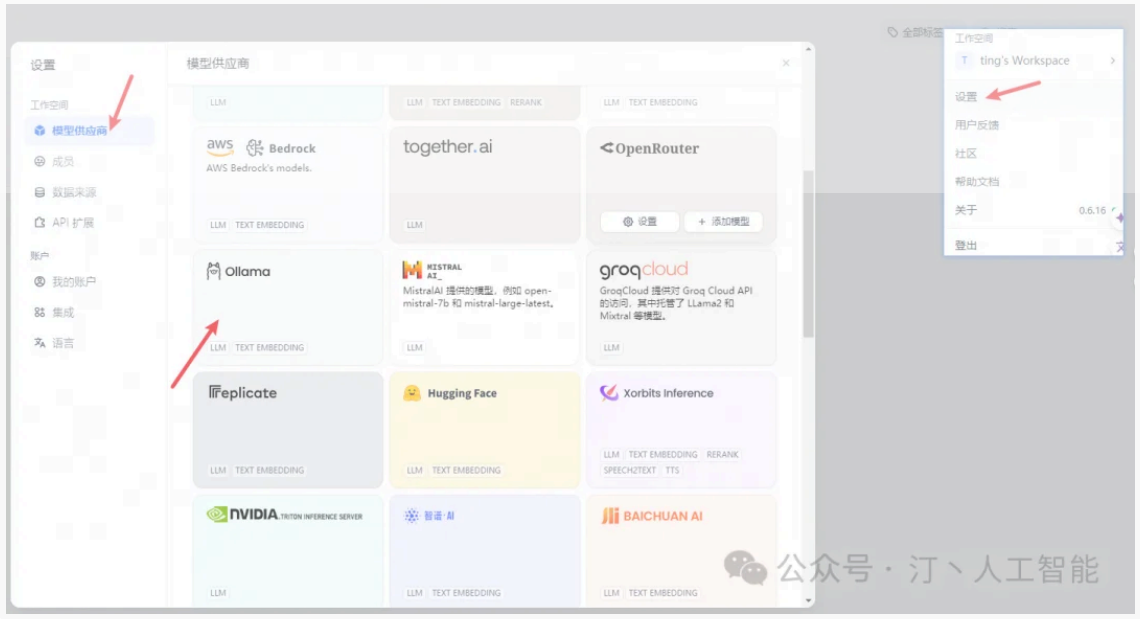
```
[Service]
ExecStart=/usr/local/bin/ollama serve
Environment="OLLAMA_HOST=0.0.0.0:7861"
Environment="OLLAMA_KEEP_ALIVE=-1"
```

运行指令

```
export OLLAMA_HOST=0.0.0.0:7861
ollama list
ollama run llama3.1

#直接添加到环境变量也可以
vim ~/.bashrc
source ~/.bashrc
```

在 设置 > 模型供应商 > Ollama 中填入：


设置
工作空间
模型供应商
成员
数据来源
API 扩展
账户
我的账户
集成
语言
模型供应商
aws Bedrock
AWS Bedrock's models.
together.ai
OpenRouter
设置
添加模型
Ollama
MISTRAL AI_
MistralAI 提供的模型，例如 open-mistral-7b 和 mistral-large-latest。
groqcloud
GroqCloud 提供对 Groq Cloud API 的访问，其中托管了 LLama2 和 Mixtral 等模型。
Replicate
Hugging Face
Xorbits Inference
NVIDIA TRITON INFERENCE SERVER
BAICHUAN AI
工作空间
ting's Workspace
设置
用户反馈
社区
帮助文档
关于
0.6.16
登出
公众号 · 汀丶人工智能




模型名称：llama3.1

基础 URL： `http://<your-ollama-endpoint-domain>:11434`

此处需填写可访问到的 Ollama 服务地址。

若 Dify 为 docker 部署，建议填写局域网 IP 地址，如：`http://10.80.2.195:11434` 或 docker 宿主机 IP 地址，如：`http://172.17.0.1:11434` 。

若为本地源码部署，可填写 `http://localhost:11434` 。

模型类型：对话

模型上下文长度：4096

模型的最大上下文长度，若不清楚可填写默认值 4096。

最大 token 上限：4096

模型返回内容的最大 token 数量，若模型无特别说明，则可与模型上下文长度保持一致。

是否支持 Vision：是

当模型支持图片理解（多模态）勾选此项，如 llava。

点击“保存”校验无误后即可在应用中使用该模型。

Embedding 模型接入方式与 LLM 类似，只需将模型类型改为 Text Embedding 即可。




如果您使用Docker部署Dify和Ollama，您可能会遇到以下错误:

```
httpconnectionpool(host=127.0.0.1, port=11434): max retries exceeded with url

httpconnectionpool(host=localhost, port=11434): max retries exceeded with url
```

这个错误是因为 Docker 容器无法访问 Ollama 服务。localhost 通常指的是容器本身，而不是主机或其他容器。要解决此问题，您需要将 Ollama 服务暴露给网络。

### 在Mac上设置环境变量

如果 `Ollama` 作为 `macOS` 应用程序运行，则应使用以下命令设置环境变量 `launchctl`：

1. 通过调用 `launchctl setenv` 设置环境变量：

```
launchctl setenv OLLAMA_HOST "0.0.0.0"
```

2. 重启Ollama应用程序。
3. 如果以上步骤无效，可以使用以下方法：

   问题是在docker内部，你应该连接到 `host.docker.internal`，才能访问docker的主机，所以将 `localhost` 替换为 `host.docker.internal` 服务就可以生效了：

```
http://host.docker.internal:11434
```

### 在Linux上设置环境变量

如果Ollama作为systemd服务运行，应该使用 `systemctl` 设置环境变量：

1. 通过调用 `systemctl edit ollama.service` 编辑systemd服务。这将打开一个编辑器。
2. 对于每个环境变量，在 `[Service]` 部分下添加一行 `Environment`：

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

3. 保存并退出。
4. 重载 `systemd` 并重启Ollama：

```
systemctl daemon-reload
systemctl restart ollama
```

公众号 · 汀丶人工智能

### 在Windows上设置环境变量

在Windows上，Ollama继承了您的用户和系统环境变量。

1. 首先通过任务栏点击Ollama退出程序
2. 从控制面板编辑系统环境变量
3. 为您的用户账户编辑或新建变量，比如 `OLLAMA_HOST`、`OLLAMA_MODELS` 等。
4. 点击OK/应用保存
5. 在一个新的终端窗口运行 `ollama`

公众号 · 汀丶人工智能

4.1.多模型对比

参考单个模型部署一样，进行再一次配置添加即可

需要注意的是添加完新的模型配置后，需要刷新dify网页，直接网页端刷新就好，新添加的模型就会加载进来



可以看到调用后模型资源消耗情况

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.86.10              Driver Version: 535.86.10    CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2-32GB           On | 00000000:00:08.0 Off |                    0 |
| N/A   35C    P0              56W / 300W |   5404MiB / 32768MiB |      0%      Default |
|                                         |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A   3359560      C   ...unners/cuda_v11/ollama_llama_server     5402MiB |
+-----------------------------------------------------------------------------+
(Dify) root@iZ8vb35kkgysmk1517hd5nZ:~# nvidia-smi
Sat Aug 10 01:05:37 2024
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.86.10              Driver Version: 535.86.10    CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2-32GB           On | 00000000:00:08.0 Off |                    0 |
| N/A   37C    P0              57W / 300W |  11922MiB / 32768MiB |      0%      Default |
|                                         |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A   3359560      C   ...unners/cuda_v11/ollama_llama_server     5534MiB |
|    0   N/A  N/A   3368392      C   ...unners/cuda_v11/ollama_llama_server     6386MiB |
+-----------------------------------------------------------------------------+
```

```
(Dify) root@iZ8vb35kkgysmk1517hd5nZ:/# ollama list
NAME            ID              SIZE    MODIFIED
llama3.1:latest 91ab477bec9d    4.7 GB  3 hours ago
qwen2:latest    e0d4e1163c58    4.4 GB  7 hours ago
```

更多LLM平台参考：

1.RAG+AI工作流+Agent：LLM框架该如何选择，全面对比MaxKB、Dify、FastGPT、RagFlow等

2.智胜未来：国内大模型+Agent应用案例精选，以及主流Agent框架开源项目推荐

3.未来已来：LLMops如何重塑AI-native新范式的运维格局[行业范式]、以及主流LLMops推荐

官方网址：https://dify.ai/zh

github地址：https://github.com/langgenius/dify/tree/main

ollama中文网址：https://ollama.fan/

ollama安装教程：https://ollama.fan/getting-started/linux/

Ollama Linux部署与应用LLama 3

**更多优质内容请关注公号：汀丶人工智能；会提供一些相关的资源和优质文章，免费获取阅读。**

**更多优质内容请关注CSDN：汀丶人工智能；会提供一些相关的资源和优质文章，免费获取阅读。**



**汀丶人工智能**

将不定期更新关于机器学习、强化学习、数据挖掘以及NLP等领域相关知识，以及分享自...

331篇原创内容

公众号

**书籍推荐：**

广告



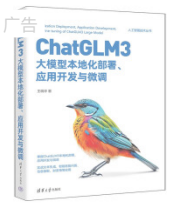
从零开始大模型开发与微调：基于PyTorch与ChatGLM（人工智能技术丛书）

京东 京东配送

¥44.5

购买

广告



ChatGLM3大模型本地化部署、应用开发与微调（人工智能技术丛书）

京东 京东配送

¥44.5

购买

广告



大模型应用开发 动手做AI Agent GPT大语言模型应用 智能代理 LangChain开发（异步图书出品）

京东 京东配送

¥44.9

购买

广告



大模型应用开发极简入门：基于GPT-4和ChatGPT LangChain LLM应用开发 蛇尾书（图灵出品）

京东 京东配送

¥29.9

购买

广告



咖哥AI系列：GPT图解+动手做Agent（京东套装2册）（异步图书出品）

京东 京东配送

¥84.8

购买

AI Agent 27

AI Agent · 目录

上一篇

告别卡顿，畅享GitHub：国内开发者必看的五大加速访问与下载技巧

下一篇

Composerize神器：自动化转换Docker运行命令至Compose配置，简化容器部署流程